

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - MEDIUM (Assignment)

COURSE CODE: CSA0904

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO2: *Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 3

Total Marks: 30

ANNEXURE A

Assignment

Code of Conduct:

I Monesh S / 192324027 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Monesh S

ANNEXURE

Assignment

1. ATM PIN Security Validator

"A nationalized bank receives ATM PINs from customers through a kiosk. Develop a Java program that accepts the PIN as a string and performs the following operations:

Remove unwanted spaces using trim()

Validate that the PIN contains exactly 4 digits using matches()

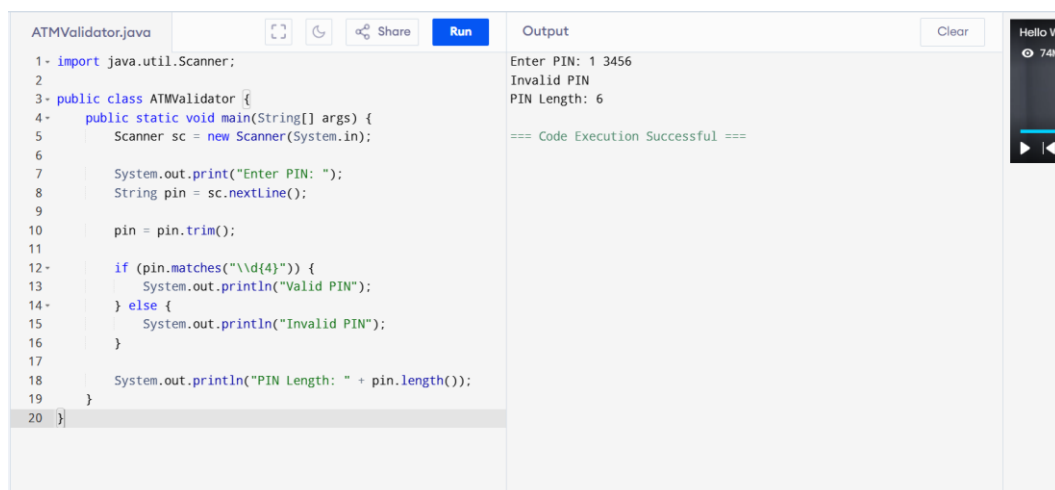
Display the PIN length using length()"

CODE:

```
import java.util.Scanner;

public class ATMValidator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter PIN: ");
        String pin = sc.nextLine();
        pin = pin.trim();
        if (pin.matches("\\d{4}")) {
            System.out.println("Valid PIN");
        } else {
            System.out.println("Invalid PIN");
        }
        System.out.println("PIN Length: " + pin.length());
    }
}
```

OUTPUT:



```
ATMValidator.java  Run  Clear  Hello Wd 74M
1- import java.util.Scanner;
2
3- public class ATMValidator {
4-     public static void main(String[] args) {
5-         Scanner sc = new Scanner(System.in);
6
7-         System.out.print("Enter PIN: ");
8-         String pin = sc.nextLine();
9
10        pin = pin.trim();
11
12-        if (pin.matches("\\d{4}")) {
13-            System.out.println("Valid PIN");
14-        } else {
15-            System.out.println("Invalid PIN");
16-        }
17
18-        System.out.println("PIN Length: " + pin.length());
19-    }
20 }
```

Output

```
Enter PIN: 1 3456
Invalid PIN
PIN Length: 6

=== Code Execution Successful ===
```

2. Employee ID Normalization System

"An HR department receives employee IDs in mixed formats such as emp1023.

Create a Java program to:

Remove spaces using trim()

Convert the alphabetic prefix to uppercase using toUpperCase()

Join the prefix and number using concat()"

CODE:

```
import java.util.Scanner;

public class EmployeeID {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Employee ID: ");
        String id = sc.nextLine();
        id = id.trim();
        String prefix = id.substring(0, 3).toUpperCase();
        String number = id.substring(3);
        System.out.println("Normalized ID: " + prefix.concat(number));
    }
}
```

OUTPUT:

EmployeeID.java	Output
<pre>1- import java.util.Scanner; 2 3- public class EmployeeID { 4- public static void main(String[] args) { 5 Scanner sc = new Scanner(System.in); 6 7 System.out.print("Enter Employee ID: "); 8 String id = sc.nextLine(); 9 10 id = id.trim(); 11 12 String prefix = id.substring(0, 3).toUpperCase(); 13 String number = id.substring(3); 14 15 String result = prefix.concat(number); 16 17 System.out.println("Normalized ID: " + result); 18 } 19 }</pre>	<pre>Enter Employee ID: emp123 Normalized ID: EMP123 === Code Execution Successful ===</pre>

3. Online Shopping Coupon Verificatio

"An e-commerce website offers the coupon code SAVE50. Write a Java program that:

Removes extra spaces using trim()

Converts the entered code to uppercase using toUpperCase()

Compares it with the original coupon using equalsIgnoreCase()"

CODE:

```
import java.util.Scanner;
```

```

public class CouponCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String coupon = "SAVE50";
        System.out.print("Enter Coupon: ");
        String input = sc.nextLine();
        input = input.trim();
        input = input.toUpperCase();
        if (coupon.equalsIgnoreCase(input))
            System.out.println("Coupon Accepted");
        else
            System.out.println("Invalid Coupon");
    }
}

```

OUTPUT:

CouponCheck.java	Output
<pre> 1- import java.util.Scanner; 2 3- public class CouponCheck { 4- public static void main(String[] args) { 5 6 Scanner sc = new Scanner(System.in); 7 8 String coupon = "SAVE50"; 9 10 System.out.print("Enter Coupon: "); 11 String input = sc.nextLine(); 12 13 input = input.trim(); 14 input = input.toUpperCase(); 15 16 if (coupon.equalsIgnoreCase(input)) 17 System.out.println("Coupon Accepted"); 18 else 19 System.out.println("Invalid Coupon"); 20 } 21 } </pre>	<pre> Enter Coupon: SAVE50 Coupon Accepted === Code Execution Successful === </pre>

4. Student Email Username Extractor

"A college portal stores student emails such as arun23@college.edu.

Develop a program that:

Finds the position of @ using indexOf()

Extracts the username using substring()

Displays the username length using length()"

CODE:

```

import java.util.Scanner;
public class EmailUser {

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter Email: ");
    String email = sc.nextLine();
    int pos = email.indexOf("@");
    String username = email.substring(0, pos);
    System.out.println("Username: " + username);
    System.out.println("Username Length: " + username.length());
}
}

```

OUTPUT:

EmailUser.java	Output
<pre> 1- import java.util.Scanner; 2 3- public class EmailUser { 4- public static void main(String[] args) { 5 6 Scanner sc = new Scanner(System.in); 7 8 System.out.print("Enter Email: "); 9 String email = sc.nextLine(); 10 11 int pos = email.indexOf("@"); 12 13 String username = email.substring(0, pos); 14 15 System.out.println("Username: " + username); 16 System.out.println("Username Length: " + username 17 .length()); 18 } </pre>	<pre> Enter Email: arun23@college.edu. Username: arun23 Username Length: 6 === Code Execution Successful === </pre>

5. Restaurant Digital Order Analyzer

"A restaurant stores orders as DOSA-3-IDLY-2-VADA-1. Create a program that:

Splits the order using split()

Converts food names to uppercase using toUpperCase()

Counts the number of ordered items using length()

CODE:

```

public class RestaurantOrder {
    public static void main(String[] args) {
        String order = "DOSA-3-IDLY-2-VADA-1";
        String[] items = order.split("-");
        System.out.println("Ordered Items:");
        for(int i=0;i<items.length;i=i+2){
            System.out.println(items[i].toUpperCase()+" - "+items[i+1]);
        }
        System.out.println("Total Food Items: "+items.length/2);}}

```

OUTPUT:

RestaurantOrder.java	Output
<pre>1- public class RestaurantOrder { 2- public static void main(String[] args) { 3- 4- String order = "DOSAs-3-IDLY-2-VADA-1"; 5- 6- String[] items = order.split("-"); 7- 8- System.out.println("Ordered Items:"); 9- 10- for(int i=0;i<items.length;i=i+2){ 11- System.out.println(items[i].toUpperCase()+" - " 12- +items[i+1]); 13- } 14- 15- System.out.println("Total Food Items: "+items.length/2); 16- } 17- }</pre>	<pre>Ordered Items: DOSAs - 3 IDLY - 2 VADA - 1 Total Food Items: 3 === Code Execution Successful ===</pre>

6. Railway Coach and Seat Parser

"Indian Railways stores ticket information as S5-43. Write a Java program that:

Splits the value using split()

Extracts the coach number using substring()

Displays the coach type using charAt()

CODE:

```
public class RailwaySeat {
    public static void main(String[] args) {
        String ticket="S5-43";
        String[] data=ticket.split("-");
        System.out.println("Coach: "+data[0]);
        System.out.println("Seat: "+data[1]);
        System.out.println("Coach Number: "+data[0].substring(1));
        System.out.println("Coach Type: "+data[0].charAt(0));
    }
}
```

OUTPUT:

RailwaySeat.java	Output
<pre>1- public class RailwaySeat { 2- public static void main(String[] args) { 3- 4- String ticket="S5-43"; 5- 6- String[] data=ticket.split("-"); 7- 8- System.out.println("Coach: "+data[0]); 9- System.out.println("Seat: "+data[1]); 10- System.out.println("Coach Number: "+data[0].substring 11- (1)); 12- System.out.println("Coach Type: "+data[0].charAt(0)); 13- } 14- }</pre>	<pre>Coach: S5 Seat: 43 Coach Number: 5 Coach Type: S === Code Execution Successful ===</pre>

7. Inventory Product Search System

"A supermarket stores product names in different letter cases. Develop a program that:

Converts product names to lowercase using toLowerCase()

Searches a product using contains()

Confirms exact matching using equalsIgnoreCase()"

CODE:

```
import java.util.Scanner;

public class ProductSearch {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        String product="Rice";
        System.out.print("Enter Product: ");
        String search=sc.nextLine();
        System.out.println("Lowercase: "+search.toLowerCase());
        System.out.println("Contains:
"+product.toLowerCase().contains(search.toLowerCase()));
        if(product.equalsIgnoreCase(search))
            System.out.println("Exact Match");
        else
            System.out.println("Not Exact Match");
    }
}
```

OUTPUT:

ProductSearch.java	Output
<pre>1- import java.util.Scanner; 2 3- public class ProductSearch { 4- public static void main(String[] args) { 5 6 Scanner sc=new Scanner(System.in); 7 8 String product="Rice"; 9 10 System.out.print("Enter Product: "); 11 String search=sc.nextLine(); 12 13 System.out.println("Lowercase: "+search.toLowerCase 14 ()); 15 16 System.out.println("Contains: "+product.toLowerCase 17 ().contains(search.toLowerCase())); 18 19 if(product.equalsIgnoreCase(search)) 20 System.out.println("Exact Match"); 21 else 22 System.out.println("Not Exact Match"); 23 } 24 }</pre>	<pre>Enter Product: RICE Lowercase: rice Contains: true Exact Match === Code Execution Successful ===</pre>

8. Grocery Shopping Cart Management System

"Add products using add()

Remove purchased products using remove()

Check whether a product exists using contains() Check how many products there in the list"

CODE:

```
import java.util.ArrayList;

public class GroceryCart {
    public static void main(String[] args) {
        ArrayList<String> cart=new ArrayList<>();
        cart.add("Rice");
        cart.add("Milk");
        cart.add("Sugar");
        cart.remove("Milk");
        System.out.println(cart);
        System.out.println("Contains Rice: "+cart.contains("Rice"));
        System.out.println("Total Products: "+cart.size());
    }
}
```

OUTPUT:

GroceryCart.java	Output
<pre>1- import java.util.ArrayList; 2 3- public class GroceryCart { 4- public static void main(String[] args) { 5 6 ArrayList<String> cart=new ArrayList<>(); 7 8 cart.add("Rice"); 9 cart.add("Milk"); 10 cart.add("Sugar"); 11 12 cart.remove("Milk"); 13 14 System.out.println(cart); 15 16 System.out.println("Contains Rice: "+cart.contains("Rice")); 17 18 System.out.println("Total Products: "+cart.size()); 19 } 20 }</pre>	<pre>[Rice, Sugar] Contains Rice: true Total Products: 2 === Code Execution Successful ===</pre>

9. Bank Token Queue Management System

"A bank serves customers based on token order. Using Queue:

Add customer tokens using offer()

Serve the next customer using poll()

Display the upcoming customer using peek()"

CODE:

```
import java.util.*;

public class BankQueue {
    public static void main(String[] args) {
        Queue<Integer> q=new LinkedList<>();
        q.offer(101);
        q.offer(102);
```

```

        q.offer(103);
        System.out.println("Served Token: "+q.poll());
        System.out.println("Next Token: "+q.peek());
        System.out.println("Queue: "+q);
    }
}

```

OUTPUT:

BankQueue.java	Output
<pre> 1- import java.util.*; 2 3- public class BankQueue { 4- public static void main(String[] args) { 5 6 Queue<Integer> q=new LinkedList<>(); 7 8 q.offer(101); 9 q.offer(102); 10 q.offer(103); 11 12 System.out.println("Served Token: "+q.poll()); 13 14 System.out.println("Next Token: "+q.peek()); 15 16 System.out.println("Queue: "+q); 17 } 18 } </pre>	<pre> Served Token: 101 Next Token: 102 Queue: [102, 103] === Code Execution Successful === </pre>

10. Railway Parcel Dispatch Tracking System

"A railway parcel office loads parcels onto a dispatch container. The last loaded parcel must be unloaded first, and parcel details must be searchable.

Requirements:

Store parcel IDs in a Stack

Store parcel ID and destination in a HashMap

Unload parcels using pop() and display the destination of the unloaded parcel"

CODE:

```

import java.util.*;

public class ParcelTracking {
    public static void main(String[] args) {
        Stack<String> stack=new Stack<>();
        HashMap<String,String> map=new HashMap<>();
        stack.push("P101");
        stack.push("P102");
        map.put("P101","Chennai");
        map.put("P102","Madurai");
        String id=stack.pop();
        System.out.println("Unloaded Parcel: "+id);
        System.out.println("Destination: "+map.get(id));
    }
}

```

OUTPUT:

ParcelTracking.java	Output
<pre>1- import java.util.*; 2 3- public class ParcelTracking { 4- public static void main(String[] args) { 5 6 Stack<String> stack=new Stack<>(); 7 HashMap<String,String> map=new HashMap<>(); 8 9 stack.push("P101"); 10 stack.push("P102"); 11 12 map.put("P101","Chennai"); 13 map.put("P102","Madurai"); 14 15 String id=stack.pop(); 16 17 System.out.println("Unloaded Parcel: "+id); 18 System.out.println("Destination: "+map.get(id)); 19 } 20 }</pre>	<pre>Unloaded Parcel: P102 Destination: Madurai === Code Execution Successful ===</pre>

11. Hospital Appointment Scheduling System

"A hospital issues appointment tokens to patients and stores patient details separately.

Requirements:

Add appointment tokens to a Queue

Store token number and patient name in a HashMap

Serve patients in token order and display the served patient details"

CODE:

```
import java.util.*;
public class HospitalAppointment {
    public static void main(String[] args) {
        Queue< Integer > q=new LinkedList<> ();
        HashMap< Integer,String > map=new HashMap<> ();
        q.offer(1);
        q.offer(2);
        map.put(1,"Arun");
        map.put(2,"Priya");
        int token=q.poll();
        System.out.println("Token: "+token);
        System.out.println("Patient: "+map.get(token));
    }
}
```

OUTPUT:

HospitalAppointment.java	Output
<pre>1- import java.util.*; 2 3- public class HospitalAppointment { 4- public static void main(String[] args) { 5 6 Queue<Integer> q=new LinkedList<>(); 7 HashMap<Integer,String> map=new HashMap<>(); 8 9 q.offer(1); 10 q.offer(2); 11 12 map.put(1,"Arun"); 13 map.put(2,"Priya"); 14 15 int token=q.poll(); 16 17 System.out.println("Token: "+token); 18 System.out.println("Patient: "+map.get(token)); 19 } 20 }</pre>	<pre>Token: 1 Patient: Arun === Code Execution Successful ===</pre>

12. University Course Enrollment Manager

"A university maintains a list of students enrolled in a course. Students who withdraw must be removed from the list."

Requirements:

Add student names to an ArrayList

Traverse the list using an Iterator

Remove withdrawn students and display the updated enrollment list"

CODE:

```
import java.util.*;

public class CourseEnrollment {
    public static void main(String[] args) {
        ArrayList<String> list=new ArrayList<>();
        list.add("Arun");
        list.add("Rahul");
        list.add("Priya");
        Iterator<String> it=list.iterator();
        while(it.hasNext()){
            String name=it.next();
            if(name.equals("Rahul"))
                it.remove();
        }
        System.out.println(list);
    }
}
```

OUTPUT:

CourseEnrollment.java	Run	Output
<pre>1- import java.util.*; 2 3- public class CourseEnrollment { 4- public static void main(String[] args) { 5 6 ArrayList<String> list=new ArrayList<>(); 7 8 list.add("Arun"); 9 list.add("Rahul"); 10 list.add("Priya"); 11 12 Iterator<String> it=list.iterator(); 13 14- while(it.hasNext()){ 15 String name=it.next(); 16 17 if(name.equals("Rahul")) 18 it.remove(); 19 } 20 21 System.out.println(list); 22 } 23 }</pre>		<pre>[Arun, Priya] === Code Execution Successful ===</pre>

13. Warehouse Damaged Item Removal

“A warehouse stores item names in an inventory list. Damaged items must be removed safely while traversing the collection.

Requirements:

Store inventory items in an ArrayList

Traverse the list using an Iterator

Remove items marked as Damaged and display the remaining inventory"

CODE:

```
import java.util.*;

public class Warehouse {

    public static void main(String[] args) {
        ArrayList<String> items=new ArrayList<>();
        items.add("Rice");
        items.add("Damaged");
        items.add("Sugar");
        Iterator<String> it=items.iterator();
        while(it.hasNext()){
            String item=it.next();
            if(item.equals("Damaged"))
                it.remove();
        }
        System.out.println(items);
    }
}
```

OUTPUT:

Warehouse.java	Output
<pre>1- import java.util.*; 2 3- public class Warehouse { 4- public static void main(String[] args) { 5 6 ArrayList<String> items=new ArrayList<>(); 7 8 items.add("Rice"); 9 items.add("Damaged"); 10 items.add("Sugar"); 11 12 Iterator<String> it=items.iterator(); 13 14- while(it.hasNext()){ 15 String item=it.next(); 16 17 if(item.equals("Damaged")) 18 it.remove(); 19 } 20 21 System.out.println(items); 22 } 23 }</pre>	<pre>[Rice, Sugar] === Code Execution Successful ===</pre>

14. Bank Token Management System

"Customers receive tokens in the order they arrive at a bank counter."

Requirements:

Add customer tokens to a Queue

Serve customers in FIFO order

Display the next customer to be served and the remaining queue"

CODE:

```
import java.util.*;

public class BankToken {

    public static void main(String[] args) {
        Queue<Integer> q=new LinkedList<>();
        q.offer(1);
        q.offer(2);
        q.offer(3);
        System.out.println("Served: "+q.poll());
        System.out.println("Next: "+q.peek());
        System.out.println("Remaining Queue: "+q);
    }
}
```

OUTPUT:

<pre>BankToken.java 1- import java.util.*; 2 3- public class BankToken { 4- public static void main(String[] args) { 5 6 Queue<Integer> q=new LinkedList<>(); 7 8 q.offer(1); 9 q.offer(2); 10 q.offer(3); 11 12 System.out.println("Served: "+q.poll()); 13 14 System.out.println("Next: "+q.peek()); 15 16 System.out.println("Remaining Queue: "+q); 17 } 18 }</pre>	<pre>Output Served: 1 Next: 2 Remaining Queue: [2, 3] === Code Execution Successful ===</pre>
--	---

15. Web Browser Back Navigation

"A web browser stores recently visited pages so that users can navigate backward."

Requirements:

Add visited page URLs using push()

Perform the Back operation using pop()

Display the current page using peek()

CODE:

```
import java.util.*;

public class BrowserBack {
    public static void main(String[] args) {
        Stack<String> pages=new Stack<>();
        pages.push("google.com");
        pages.push("youtube.com");
        pages.push("github.com");
        System.out.println("Back From: "+pages.pop());
        System.out.println("Current Page: "+pages.peek());
        System.out.println("Pages: "+pages);
    }
}
```

OUTPUT:

BrowserBack.java	Output
<pre>1- import java.util.*; 2 3- public class BrowserBack { 4- public static void main(String[] args) { 5 6 Stack<String> pages=new Stack<>(); 7 8 pages.push("google.com"); 9 pages.push("youtube.com"); 10 pages.push("github.com"); 11 12 System.out.println("Back From: "+pages.pop()); 13 14 System.out.println("Current Page: "+pages.peek()); 15 16 System.out.println("Pages: "+pages); 17 } 18 }</pre>	<pre>Back From: github.com Current Page: youtube.com Pages: [google.com, youtube.com] === Code Execution Successful ===</pre>

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification